

ĐẠI HỌC QUỐC GIA HÀ NỘI
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ



Nguyễn Quế Sơn
Nguyễn Hải An

TẤN CÔNG GIẢI ẨN DANH TRÊN DỮ LIỆU THỬA

*(Mô phỏng tấn công Netflix Prize bằng thuật toán Scoreboard-RH trên dữ liệu
MovieLens)*

BÁO CÁO BÀI TẬP LỚN CUỐI KỲ

Học phần: An ninh di động

HÀ NỘI - 2026

ĐẠI HỌC QUỐC GIA HÀ NỘI
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ

Nguyễn Quế Sơn

Nguyễn Hải An

TÂN CÔNG GIẢI ẨN DANH TRÊN DỮ LIỆU THỬA

*(Mô phỏng tấn công Netflix Prize bằng thuật toán Scoreboard-RH trên dữ liệu
MovieLens)*

BÁO CÁO BÀI TẬP LỚN CUỐI KỲ

Học phần: An ninh di động

Giảng viên hướng dẫn: Lê Thị Hời

HÀ NỘI - 2026

LỜI CAM ĐOAN

Chúng em xin cam đoan báo cáo bài tập lớn “*Tấn công giải ẩn danh trên dữ liệu thưa*” là do chúng em tự tìm hiểu, nghiên cứu và thực hiện dựa trên công trình khoa học gốc của Narayanan và Shmatikov. Toàn bộ kết quả thực nghiệm trình bày trong báo cáo là kết quả thật, thu được trực tiếp từ việc chạy ứng dụng mô phỏng do chúng em xây dựng trên tập dữ liệu công khai MovieLens 100k. Mọi thực nghiệm chỉ thao tác trên dữ liệu công khai phục vụ mục đích học thuật, không nhằm vào bất kỳ cá nhân hay hệ thống thật nào. Các tài liệu tham khảo đều đã được liệt kê đầy đủ ở phần Tài liệu tham khảo. Chúng em xin hoàn toàn chịu trách nhiệm về tính trung thực và chính xác của toàn bộ nội dung trong báo cáo này.

Nhóm tác giả

Nguyễn Quế Sơn

Nguyễn Hải An

LỜI CẢM ƠN

Chúng em xin gửi lời cảm ơn chân thành đến cô Lê Thị Hợi, giảng viên học phần An ninh di động, đã hướng dẫn, định hướng và cung cấp kiến thức nền tảng giúp chúng em hoàn thành báo cáo bài tập lớn này. Chúng em cũng xin cảm ơn các thầy cô Khoa Công nghệ thông tin, Trường Đại học Công nghệ - Đại học Quốc gia Hà Nội vì những kiến thức quý báu đã truyền đạt trong suốt quá trình học tập.

Do thời gian và kiến thức còn hạn chế, báo cáo không tránh khỏi những thiếu sót. Chúng em rất mong nhận được các góp ý của cô để hoàn thiện hơn.

MỤC LỤC

Lời cam đoan	
Lời cảm ơn	
Mục lục	i
Tóm tắt	iv
Danh sách từ viết tắt	v
Danh mục hình vẽ	vi
Danh mục bảng biểu	vii
Mở đầu	1
CHƯƠNG 1. Cơ sở lý thuyết	3
1.1. Bài toán ẩn danh hóa micro-data	3
1.2. Các phương pháp ẩn danh truyền thống và hạn chế	3
1.2.1. k-anonymity	3
1.2.2. l-diversity, t-closeness và giới hạn chung	4
1.2.3. Các tiền lệ rò rỉ thực tế	5
1.3. Độ thừa và độ tương tự	5
1.4. Mô hình tấn công và định nghĩa rò rỉ riêng tư	6
1.4.1. Tri thức bổ trợ	6
1.4.2. Hai kịch bản rò rỉ	6
1.5. Thuật toán Scoreboard và Scoreboard-RH	6
1.5.1. Trọng số theo độ hiếm	7
1.5.2. Cộng gộp tương tự rating và thời gian	7
1.5.3. Tiêu chí eccentricity	7
1.6. Phân tích: cần bao nhiêu tri thức bổ trợ?	8
1.7. Tình huống điển hình: tập Netflix Prize	8
1.7.1. Kết quả giải ẩn danh	8
1.7.2. Tấn công thực tế bằng dữ liệu IMDb	9
1.7.3. Tính riêng tư xuôi (forward privacy)	9
1.8. Liên hệ với an ninh di động	9

CHƯƠNG 2. Phân tích bài toán và thiết kế hệ thống	11
2.1. Lựa chọn tập dữ liệu	11
2.2. Kiến trúc hệ thống	11
2.3. Mô hình dữ liệu	12
2.4. Luồng tấn công	13
CHƯƠNG 3. Cài đặt thuật toán và nâng cấp ứng dụng	14
3.1. Cài đặt hàm cho điểm	14
3.2. Cài đặt tiêu chí eccentricity	15
3.3. Các nâng cấp so với bản demo ban đầu	15
3.3.1. Sửa hàm cho điểm cho đúng công thức gốc	15
3.3.2. Bổ sung chế độ “không dùng ngày”	15
3.3.3. Phơi bày trọng số độ hiếm trên giao diện	15
3.3.4. Sinh nạn nhân thật và mô phỏng nhiễu	16
3.3.5. Module thực nghiệm định lượng	16
3.4. Công nghệ và môi trường	16
CHƯƠNG 4. Thực nghiệm và kết quả	17
4.1. Môi trường thực nghiệm	17
4.2. Một phiên tấn công qua giao diện	17
4.2.1. Tổng quan hệ thống	17
4.2.2. Nạp tri thức hỗ trợ	17
4.2.3. Kết quả định danh	17
4.2.4. Tính bền vững trước thông tin nhiễu	18
4.3. Kiểm chứng trên dòng lệnh	18
4.4. Kết quả định lượng	19
4.5. Phân tích kết quả	21
4.6. Tình huống thất bại và ý nghĩa của ngưỡng	23
CHƯƠNG 5. Thảo luận, phòng vệ và kết luận	25
5.1. Ý nghĩa về quyền riêng tư	25
5.2. Các biện pháp phòng vệ	25
5.2.1. Vì sao k-anonymity không đủ	25
5.2.2. Riêng tư vi phân (Differential Privacy)	25
5.2.3. Các biện pháp thực tiễn khác	25
5.3. Hạn chế của nghiên cứu	26

5.4. Kết luận	26
TÀI LIỆU THAM KHẢO	27
PHỤ LỤC	29

TÓM TẮT

Tóm tắt: Báo cáo nghiên cứu và mô phỏng lại lớp tấn công giải ẩn danh (de-anonymization) trên các tập dữ liệu thưa, dựa trên công trình kinh điển “*Robust De-anonymization of Large Sparse Datasets*” của Narayanan và Shmatikov - cũng chính là cuộc tấn công đã phá vỡ tính ẩn danh của tập dữ liệu Netflix Prize. Chúng em phân tích mô hình hình thức của bài toán (độ thưa, độ tương tự, định nghĩa rò rỉ riêng tư), thuật toán Scoreboard-RH cùng tiêu chí *eccentricity*, sau đó xây dựng một ứng dụng web hoàn chỉnh (FastAPI + React) mô phỏng kẻ tấn công trên tập dữ liệu công khai MovieLens 100k.

Bằng thực nghiệm thật trên 943 người dùng và 100.000 lượt đánh giá, chúng em chứng minh: chỉ với 8 bộ phim + ngày xem, thuật toán định danh đúng 98% người dùng; ngay cả khi không có thông tin ngày vẫn đạt 93%; và với thông tin nhiễu (± 1 điểm, ± 14 ngày) vẫn đạt 72% - khẳng định tính *bền vững* của tấn công. Khi ưu tiên các bộ phim hiếm, chỉ 2 phim đã đủ định danh 59% người dùng. Báo cáo cũng so sánh, tối ưu và nâng cấp ứng dụng demo cho đúng công thức trong bài báo gốc, rồi thảo luận các biện pháp phòng vệ (k-anonymity, riêng tư vi phân, hạn chế công bố micro-data).

Từ khóa: giải ẩn danh, dữ liệu thưa, quyền riêng tư, Netflix Prize, MovieLens, Scoreboard-RH, *eccentricity*.

DANH SÁCH TỪ VIẾT TẮT

Từ viết tắt	Giải nghĩa
PII	Personally Identifiable Information - Thông tin định danh cá nhân
aux	Auxiliary information - Tri thức bổ trợ kẻ tấn công nắm giữ
supp	Support - Tập thuộc tính khác rỗng của một bản ghi / số người đánh giá một phim
RH	Robust, Histogram - Biểu thể bền vững của thuật toán Scoreboard
ecc	Eccentricity - Độ “nổi bật” của ứng viên khớp nhất so với phần còn lại
DP	Differential Privacy - Riêng tư vi phân
API	Application Programming Interface
REST	Representational State Transfer
CSV	Comma-Separated Values
ORM	Object-Relational Mapping
UI	User Interface - Giao diện người dùng

DANH MỤC HÌNH VẼ

Hình 2.1	Kiến trúc ba lớp của hệ thống mô phỏng tấn công giải ẩn danh . . .	12
Hình 4.1	Màn hình chính: thống kê độ thưa và bảng điều khiển kẻ tấn công . .	18
Hình 4.2	Bảng điều khiển sau khi nạp aux: mỗi phim hiển thị support và trọng số độ hiếm	19
Hình 4.3	Định danh thành công User #373 với eccentricity 3,8333	20
Hình 4.4	Lịch sử riêng tư của nạn nhân bị phơi bày hoàn toàn (233 lượt xem) .	20
Hình 4.5	Bảng xếp hạng ứng viên: hạng nhất nổi bật hẳn so với phần còn lại .	21
Hình 4.6	Vấn định danh đúng dù aux có nhiễu (± 1 điểm, ± 14 ngày)	22
Hình 4.7	Một phiên tấn công thật chạy trên dòng lệnh (nạn nhân #200)	22
Hình 4.8	Kết xuất (log) phiên chạy thực nghiệm trên 300 nạn nhân tại giao diện dòng lệnh, 4 kích bản $\times$ 5 mức số phim	23
Hình 4.9	Tỉ lệ định danh thành công theo lượng tri thức bổ trợ	23
Hình 4.10	Eccentricity trung bình theo lượng tri thức bổ trợ (đường nét đứt là ngưỡng $\phi = 1,5$)	24

DANH MỤC BẢNG BIỂU

Bảng 2.1	Thông số tập dữ liệu MovieLens 100k	11
Bảng 2.2	Đối chiếu tập Netflix gốc và proxy MovieLens 100k	12
Bảng 2.3	Các điểm cuối (endpoint) REST của backend	12
Bảng 4.1	Tỉ lệ định danh đúng (%) theo số phim biết trước và kịch bản	20

MỞ ĐẦU

Bối cảnh và động lực

Ngày nay, các tổ chức thường xuyên công bố những tập dữ liệu lớn về hành vi cá nhân - lịch sử mua hàng, đánh giá phim, lượt nghe nhạc, vị trí di chuyển - phục vụ nghiên cứu, khuyến nghị và khai phá dữ liệu. Để bảo vệ người dùng, các đơn vị này thường “ẩn danh hóa” dữ liệu bằng cách loại bỏ tên, số định danh và các thông tin nhận dạng trực tiếp. Một quan niệm phổ biến cho rằng: chỉ cần gỡ bỏ thông tin định danh là dữ liệu đã an toàn.

Năm 2008, Arvind Narayanan và Vitaly Shmatikov đã bác bỏ quan niệm đó. Trong công trình “*Robust De-anonymization of Large Sparse Datasets*” [1], hai tác giả chứng minh rằng với một tập dữ liệu thưa và nhiều chiều (như lịch sử xem phim), kẻ tấn công chỉ cần biết một vài thông tin bên lề về nạn nhân là đủ để xác định chính xác bản ghi của người đó và phơi bày toàn bộ phần dữ liệu còn lại. Họ áp dụng phương pháp này lên tập dữ liệu Netflix Prize - gồm hơn 100 triệu lượt đánh giá phim của 500.000 thuê bao - và định danh thành công nhiều cá nhân, làm lộ cả những thông tin nhạy cảm như xu hướng chính trị hay sở thích cá nhân. Kết quả này về sau dẫn tới một vụ kiện và khiến Netflix hủy bỏ cuộc thi Netflix Prize lần thứ hai.

Đây là một trong những minh chứng nền tảng cho thấy “ẩn danh hóa” không đồng nghĩa với “riêng tư”, và là bài học cốt lõi trong lĩnh vực an toàn dữ liệu nói chung cũng như an ninh trên thiết bị, ứng dụng di động nói riêng - nơi dữ liệu hành vi người dùng được thu thập với mật độ dày đặc.

Mục tiêu

Báo cáo đặt ra các mục tiêu sau:

- Nghiên cứu và trình bày lại mô hình tấn công giải ẩn danh của Narayanan-Shmatikov: nền tảng lý thuyết (độ thưa, độ tương tự, định nghĩa rò rỉ riêng tư) và thuật toán Scoreboard-RH.
- Xây dựng, tối ưu và nâng cấp một ứng dụng web mô phỏng cuộc tấn công trên tập dữ liệu công khai MovieLens 100k, đóng vai proxy cho tập Netflix.

- Thực nghiệm định lượng thật để kiểm chứng các kết luận của bài báo gốc: cần bao nhiêu tri thức hỗ trợ để định danh thành công, và mức độ bền vững của tấn công trước thông tin sai lệch.
- Thảo luận ý nghĩa về quyền riêng tư và các biện pháp phòng vệ.

Phạm vi và phương pháp

Toàn bộ thực nghiệm chỉ thao tác trên tập dữ liệu công khai MovieLens 100k, trong môi trường cục bộ, không nhằm vào bất kỳ cá nhân hay hệ thống thật nào. Mọi số liệu, biểu đồ và ảnh chụp màn hình trong báo cáo đều là kết quả thật, sinh ra trực tiếp từ việc chạy ứng dụng do chúng em phát triển. Ngôn ngữ cài đặt là Python (engine tấn công, FastAPI) và JavaScript/React (giao diện).

Cấu trúc báo cáo

Chương 1 trình bày cơ sở lý thuyết về bài toán giải ẩn danh và thuật toán Scoreboard-RH. Chương 2 phân tích bài toán và thiết kế kiến trúc hệ thống mô phỏng. Chương 3 mô tả chi tiết việc cài đặt thuật toán cùng những tối ưu, nâng cấp so với bản demo ban đầu. Chương 4 trình bày kịch bản demo và các kết quả thực nghiệm định lượng. Chương 5 thảo luận biện pháp phòng vệ, nêu hạn chế và kết luận.

CHƯƠNG 1

CƠ SỞ LÝ THUYẾT

Chương này trình bày nền tảng lý thuyết của tấn công giải ẩn danh: từ hạn chế của các phương pháp ẩn danh truyền thống, mô hình hình thức hóa bài toán, thuật toán Scoreboard-RH và phân tích lý thuyết về lượng tri thức cần thiết, đến tình huống điển hình Netflix Prize và mối liên hệ với an ninh di động. Nội dung chương dựa chủ yếu trên công trình gốc của Narayanan và Shmatikov [1], có bổ sung từ các công trình liên quan.

1.1. Bài toán ẩn danh hóa micro-data

Micro-data là dữ liệu ở mức từng cá nhân: mỗi bản ghi mô tả một người và gồm nhiều thuộc tính. Khác với dữ liệu thống kê tổng hợp (chỉ công bố các đại lượng gộp như trung bình, tổng), *micro-data* giữ lại bản ghi riêng của từng người ngay cả sau khi đã “ẩn danh hóa”. Các tập như lịch sử mua hàng, hồ sơ y tế, log tìm kiếm, dấu vết di chuyển GPS hay lịch sử xem phim đều là *micro-data*.

Nhu cầu công bố *micro-data* ngày càng lớn: phục vụ nghiên cứu y học, huấn luyện hệ khuyến nghị, minh bạch hóa dữ liệu công (“open data”). Để bảo vệ người dùng, đơn vị công bố thường *loại bỏ thông tin định danh trực tiếp* (Personally Identifiable Information - PII) như họ tên, số căn cước, địa chỉ. Câu hỏi trung tâm của chương này là: *liệu việc gỡ bỏ PII có thực sự bảo vệ được quyền riêng tư hay không?*

Câu trả lời là không. Hai đặc trưng của *micro-data* hành vi - *số chiều cao* (high dimensionality) và *độ thưa* (sparsity) - khiến mỗi bản ghi gần như là một “dấu vân tay” duy nhất, có thể bị liên kết ngược về danh tính thật chỉ với một lượng nhỏ thông tin bên lề.

1.2. Các phương pháp ẩn danh truyền thống và hạn chế

1.2.1. *k-anonymity*

Mô hình bảo vệ kinh điển nhất là *k-anonymity* [4]. Đơn vị công bố chia thuộc tính thành hai nhóm:

- *Quasi-identifier* (QID): các thuộc tính tuy không định danh trực tiếp nhưng có thể liên kết với nguồn dữ liệu khác để truy ra danh tính (ví dụ: tuổi, giới tính, mã vùng)

bầu chính).

- *Thuộc tính nhạy cảm*: thông tin cần bảo vệ (ví dụ: bệnh án, mức lương).

Dữ liệu được *khái quát hóa* (generalization) và *nép* (suppression) sao cho mỗi tổ hợp giá trị QID xuất hiện ở ít nhất k bản ghi. Như vậy, một bản ghi không thể bị phân biệt khỏi $k - 1$ bản ghi khác cùng nhóm QID.

Động lực của k -anonymity đến từ một vụ tấn công thực tế: năm 1997, L. Sweeney liên kết tập dữ liệu xuất viện “ẩn danh” của bang Massachusetts (gồm mã vùng, ngày sinh, giới tính) với danh sách cử tri công khai, và truy ra được *bệnh án của chính thống đốc bang* [4]. Chỉ ba thuộc tính tưởng như vô hại đã đủ định danh phần lớn dân số Hoa Kỳ.

1.2.2. *l*-diversity, *t*-closeness và giới hạn chung

k -anonymity bảo vệ khỏi việc *định danh bản ghi*, nhưng không bảo vệ giá trị nhạy cảm. Nếu cả k bản ghi trong một nhóm QID đều có cùng giá trị nhạy cảm (tấn công đồng nhất - homogeneity attack), kẻ tấn công vẫn suy ra được giá trị đó. Các cải tiến ra đời để vá lỗ hổng này:

- ***l*-diversity** [5]: yêu cầu mỗi nhóm QID chứa ít nhất l giá trị nhạy cảm “đủ khác biệt”.
- ***t*-closeness** [6]: yêu cầu phân phối thuộc tính nhạy cảm trong mỗi nhóm gần với phân phối toàn cục (khoảng cách $\leq t$).

Tuy nhiên, toàn bộ họ phương pháp này có một giả định nền tảng bị phá vỡ khi áp dụng cho micro-data hành vi: chúng giả định có thể *tách trước* tập QID khỏi thuộc tính nhạy cảm. Với dữ liệu như lịch sử xem phim, giả định này sai về bản chất:

- **Mọi thuộc tính đều có thể là QID**. Kẻ tấn công có thể biết *bất kỳ* bộ phim nào về nạn nhân; không thể chỉ định trước một tập QID cố định.
- **Thất bại trên dữ liệu nhiều chiều**. Khi số thuộc tính lên tới hàng nghìn, để mọi tổ hợp xuất hiện $\geq k$ lần phải khái quát hóa mạnh tới mức dữ liệu mất hết giá trị sử dụng [1].

Chính vì vậy, tập Netflix Prize - như sẽ thấy - hoàn toàn *không* k -ẩn danh với bất kỳ $k > 1$ nào.

1.2.3. Các tiền lệ rò rỉ thực tế

Ngoài vụ Massachusetts, hai vụ việc nổi tiếng khác minh họa rủi ro:

- **AOL (2006):** AOL công bố 20 triệu truy vấn tìm kiếm “ẩn danh” (chỉ thay tên người dùng bằng số). Phóng viên New York Times đã truy ra danh tính người dùng số 4417749 chỉ từ nội dung các truy vấn của bà [7].
- **MovieLens (2006):** Frankowski và cộng sự liên kết các bài đăng công khai trên diễn đàn MovieLens với hồ sơ đánh giá nội bộ, cho thấy ngay cả “đề cập công khai” về phim cũng đủ gây rò rỉ [11].

1.3. Độ thưa và độ tương tự

Quan sát then chốt của Narayanan-Shmatikov là các tập dữ liệu hành vi thực tế đều thưa: mỗi người chỉ tương tác với một phần rất nhỏ trong không gian thuộc tính khổng lồ. Trên tập Netflix, mỗi thuê bao chỉ đánh giá vài chục đến vài trăm phim trong số hàng chục nghìn phim. Hệ quả là mỗi bản ghi gần như duy nhất: rất khó tìm được hai người có cùng tổ hợp phim và điểm. Chính sự duy nhất này là mầm mống của rò rỉ riêng tư.

Gọi $\text{supp}(r)$ là *support* của bản ghi r - tập các thuộc tính khác rỗng (các phim mà người đó đã đánh giá). Phân phối số người đánh giá mỗi phim tuân theo quy luật “đuôi dài” (long-tail) hay luật lũy thừa: một số ít phim bom tấn được rất nhiều người xem, còn phần lớn phim chỉ có ít người xem. Các phim hiếm ở phần đuôi mang giá trị định danh cao.

Để đo mức “giống nhau” giữa hai bản ghi, ta dùng hàm tương tự $\text{Sim}(r_1, r_2) \in [0, 1]$. Một cách hình thức, bài báo định nghĩa tính thưa như sau:

Định nghĩa (Độ thưa). Một cơ sở dữ liệu D được gọi là (ϵ, δ) -thưa theo độ tương tự Sim nếu

$$\Pr_r [\exists r' \neq r : \text{Sim}(r, r') > \epsilon] \leq \delta \quad (1.1)$$

nghĩa là xác suất một bản ghi ngẫu nhiên có “hàng xóm gần” với độ tương tự vượt ϵ là không quá δ .¹

Bài báo chứng minh tập Netflix thưa áp đảo: với đại đa số bản ghi, không tồn tại

¹Bài báo gốc viết điều kiện trong ngoặc dưới dạng $\text{Sim}(r, r') > \epsilon \forall r' \neq r$. Ở đây chúng em dùng lượng từ $\exists$ cho đúng với ngữ nghĩa mà chính bài báo mô tả - “không tồn tại bản ghi nào khác đủ giống” - tức xác suất một bản ghi có ít nhất một hàng xóm gần là rất nhỏ.

bản ghi nào khác có độ tương tự trên 0,5 trong toàn bộ 500.000 bản ghi - ngay cả khi chỉ xét tập phim đã xem mà bỏ qua điểm số và ngày.

1.4. Mô hình tấn công và định nghĩa rò rỉ riêng tư

1.4.1. Tri thức bổ trợ

Kẻ tấn công có một tập $aux = Aux(r)$ - thông tin bên lề về nạn nhân r , gồm giá trị của một tập con nhỏ các thuộc tính. Điểm mấu chốt làm nên tên gọi “robust”: aux không cần chính xác - điểm có thể sai lệch, ngày có thể chỉ biết gần đúng, thậm chí một vài mục hoàn toàn sai. Một cách hình thức, Aux được mô hình hóa như một hàm xác suất ánh xạ bản ghi thật sang phiên bản nhiễu mà kẻ tấn công nắm giữ. Kẻ tấn công có thể thu thập aux từ trò chuyện đời thường, mạng xã hội, hay đánh giá công khai của nạn nhân trên một dịch vụ khác.

1.4.2. Hai kịch bản rò rỉ

Bài báo phân biệt hai dạng vi phạm riêng tư:

- **Giải ẩn danh “đoán tốt nhất” (best-guess):** thuật toán đưa ra một bản ghi r' và khẳng định đó là nạn nhân. Vi phạm xảy ra nếu $\text{Sim}(r, r') \geq \theta$ với xác suất cao.
- **Giải ẩn danh “danh sách nghi phạm” (lineup):** thuật toán đưa ra một tập nhỏ ứng viên kèm phân phối xác suất, tương tự khái niệm ẩn danh trong mạng trộn (mix network). Mức rò rỉ được đo bằng *entropy* của phân phối: kẻ tấn công cần thêm bao nhiêu bit thông tin để xác định duy nhất nạn nhân.

Vì dữ liệu thừa, bản ghi thỏa $\text{Sim}(r, r') \geq \theta$ gần như chắc chắn là duy nhất - nên danh sách nghi phạm thường thu về đúng một người. Khi đó kẻ tấn công học được *toàn bộ* thuộc tính của bản ghi đó, kể cả các thuộc tính nhạy cảm mà ban đầu hẳn không biết.

1.5. Thuật toán Scoreboard và Scoreboard-RH

Thuật toán giải ẩn danh được mô tả như một *khuôn mẫu* (template) gồm ba thành phần:

1. **Hàm cho điểm** $\text{Score}(aux, r')$: gán cho mỗi bản ghi ứng viên một điểm thể hiện mức khớp với aux .
2. **Tiêu chí khớp** (matching): từ tập điểm, quyết định có một khớp hay không.
3. **Chọn bản ghi** (selection): trả về bản ghi “đoán tốt nhất” hoặc phân phối xác suất.

Thuật toán cơ sở Scoreboard gán điểm bằng độ tương tự nhỏ nhất giữa ứng viên và aux, rồi chọn nếu điểm vượt một hằng số α . Cách này không bền vững: chỉ cần một thuộc tính trong aux hoàn toàn sai là điểm sụp đổ.

Phiên bản bền vững Scoreboard-RH (Robust, Histogram) cải tiến hàm cho điểm theo hai heuristic quan trọng:

1.5.1. Trọng số theo độ hiếm

Các thuộc tính hiếm được gán trọng số lớn hơn. Biết nạn nhân xem một bộ phim nghệ thuật ít người biết có giá trị định danh cao hơn nhiều so với biết họ xem một bom tấn. Trọng số định nghĩa:

$$wt(i) = \frac{1}{\log |\text{supp}(i)|} \quad (1.2)$$

với $|\text{supp}(i)|$ là số người đã đánh giá phim i . Phim càng ít người xem, $wt(i)$ càng lớn.

1.5.2. Cộng gộp tương tự rating và thời gian

Với mỗi phim i trong aux, điểm tích lũy theo cả độ lệch điểm số và độ lệch thời gian, dùng hàm mũ suy giảm:

$$\text{Score}(aux, r') = \sum_{i \in \text{supp}(aux)} wt(i) \left(e^{-\frac{|\rho_i - \rho'_i|}{\rho_0}} + e^{-\frac{|d_i - d'_i|}{d_0}} \right) \quad (1.3)$$

trong đó ρ_i, d_i là điểm và thời điểm trong aux; ρ'_i, d'_i là của ứng viên r' ; ρ_0, d_0 là hằng số co giãn (bài báo chọn $\rho_0 = 1,5$ và $d_0 = 30$ ngày). Hàm mũ khiến các khớp “gần đúng” vẫn được tính điểm dương - đây là chìa khóa của tính bền vững: một phim vẫn đóng góp ngay cả khi ngày xem sai lệch nhiều.

1.5.3. Tiêu chí eccentricity

Để quyết định ứng viên điểm cao nhất có thực sự là nạn nhân hay chỉ là trùng hợp, thuật toán dùng eccentricity: khoảng cách giữa điểm cao nhất và điểm cao nhì, chuẩn hóa theo độ lệch chuẩn của toàn bộ phân phối điểm:

$$\text{ecc} = \frac{\max_1 - \max_2}{\sigma} \quad (1.4)$$

Nếu $\text{ecc} > \phi$ (ngưỡng, bài báo chọn $\phi = 1,5$) thì ứng viên cao nhất “nổi bật” hẳn so với đám đông và được kết luận là nạn nhân; ngược lại, thuật toán *từ chối định danh* để tránh dương tính giả. Đây là cơ chế tự kiểm soát sai số quan trọng: nó cho phép thuật toán *biết khi nào nó không biết*, nhờ đó mỗi lần khẳng định đều có độ tin cậy cao.

1.6. Phân tích: cần bao nhiêu tri thức hỗ trợ?

Một đóng góp lý thuyết quan trọng của bài báo là định lượng lượng tri thức hỗ trợ tối thiểu để tấn công thành công. Kết quả chính có thể phát biểu trực giác như sau: với một cơ sở dữ liệu N bản ghi đủ thưa, kẻ tấn công chỉ cần biết khoảng

$$m \geq \frac{\log N}{-\log(1 - \epsilon)} \quad (1.5)$$

thuộc tính của nạn nhân (mỗi thuộc tính khớp tới độ chính xác $1 - \epsilon$) là đủ để định danh duy nhất bản ghi với xác suất cao. Điều đáng chú ý là m chỉ tăng theo logarit của kích thước cơ sở dữ liệu: dù dữ liệu lớn gấp nghìn lần, lượng tri thức cần thiết hầu như không đổi. Với dữ liệu *thưa*, m còn nhỏ hơn nữa - đây là lý do chỉ cần 5-8 thuộc tính là đủ để giải ẩn danh hầu hết bản ghi trong các tập thực tế.

Bài báo cũng chứng minh tính bền vững một cách hình thức: ngay cả khi dữ liệu công bố đã bị thêm nhiễu (sanitization) và một phần aux bị sai, thuật toán vẫn không tạo ra khớp giả với xác suất cao, miễn là dữ liệu đủ thưa. Trực giác là nhiễu nhẹ không thể “xóa nhòa” tính duy nhất vốn rất mạnh của các bản ghi thưa.

1.7. Tình huống điển hình: tập Netflix Prize

Năm 2006, Netflix công bố tập dữ liệu gồm hơn 100 triệu lượt đánh giá của khoảng 500.000 thuê bao (đã loại bỏ thông tin định danh) để tổ chức cuộc thi trị giá 1 triệu USD cải tiến hệ khuyến nghị. Netflix khẳng định dữ liệu an toàn vì “mọi thông tin định danh đã được gỡ bỏ”. Tập công bố thực chất chỉ là một mẫu nhỏ (dưới 1/10) của cơ sở dữ liệu gốc.

1.7.1. Kết quả giải ẩn danh

Áp dụng Scoreboard-RH, Narayanan-Shmatikov chứng minh điều ngược lại. Các kết quả chính (sẽ được chúng em tái hiện ở Chương 4 trên MovieLens):

- Biết 8 phim (trong đó 2 phim có thể sai) cùng ngày xem với sai số 14 ngày là đủ định danh duy nhất 99% thuê bao.
- Chỉ với 2 phim cùng ngày (sai số 3 ngày), 68% bản ghi đã bị định danh.
- Ngay cả khi không biết ngày, 84% thuê bao vẫn bị định danh nếu kẻ tấn công biết 6 trong 8 phim nằm ngoài top 500 phim phổ biến nhất.

1.7.2. Tấn công thực tế bằng dữ liệu IMDb

Để chứng minh tính khả thi, các tác giả lấy tri thức bổ trợ từ đánh giá công khai trên IMDb: nhiều người dùng đánh giá phim công khai trên IMDb (thường dưới tên thật) đồng thời là thuê bao Netflix. Chỉ với một mẫu nhỏ khoảng 50 người dùng IMDb, thuật toán định danh được bản ghi Netflix của họ với eccentricity lên tới 28 và 15 độ lệch chuẩn - những khớp cực kỳ chắc chắn, gần như không thể là dương tính giả.

Hậu quả về riêng tư là nghiêm trọng: từ lịch sử Netflix bị phơi bày, có thể suy ra các thông tin *nhạy cảm không hề công khai trên IMDb* - như quan điểm chính trị (qua các phim tài liệu đã xem), tôn giáo, hay xu hướng tính dục của nạn nhân. Đây không phải vi phạm giả định: chính những suy luận này về sau dẫn tới một vụ kiện tập thể chống Netflix [13] (viện dẫn Đạo luật Bảo vệ Riêng tư Video - VPPA [10]) và khiến Netflix hủy bỏ cuộc thi Netflix Prize thứ hai.

1.7.3. Tính riêng tư xuôi (*forward privacy*)

Một hệ quả tinh tế: một khi một mẫu dữ liệu bị liên kết với danh tính thật, mọi liên kết về *sau* đều dễ dàng hơn - kể cả khi nạn nhân tạo một định danh ảo hoàn toàn mới. Điều này tương tự khái niệm “forward secrecy” trong mật mã, nhưng ở chiều ngược lại và bất lợi cho người dùng: dữ liệu hành vi một khi đã rò rỉ thì không thể “thu hồi”.

1.8. Liên hệ với an ninh di động

Bài học của Narayanan-Shmatikov không giới hạn ở dữ liệu phim, mà đặc biệt thời sự với an ninh trên thiết bị và ứng dụng di động - nơi dữ liệu hành vi người dùng được thu thập với mật độ dày đặc và cũng mang đặc tính thưa, nhiều chiều:

- **Dấu vết di chuyển (mobility traces).** de Montjoye và cộng sự [8] chứng minh rằng chỉ cần 4 điểm thời gian-không gian (ví dụ vị trí gần đúng tại 4 thời điểm) là đủ để định danh duy nhất 95% người dùng trong một tập 1,5 triệu người - cùng bản chất “thưa nên duy nhất” như tập Netflix. Vị trí được điện thoại di động ghi lại liên tục, khiến đây là rủi ro trực tiếp.
- **Siêu dữ liệu giao dịch.** Cũng nhóm này [9] cho thấy 4 giao dịch thẻ tín dụng là đủ định danh 90% người dùng - dữ liệu thường được các ứng dụng ví điện tử thu thập.
- **Vân tay thiết bị và cảm biến.** Tập hợp các thuộc tính tương tự vô hại của một thiết bị di động (danh sách ứng dụng đã cài, phiên bản hệ điều hành, đặc trưng cảm biến

gia tốc, danh sách font, độ phân giải) cũng tạo thành một “dấu vân tay” thừa và nhiều chiều, cho phép theo dõi xuyên ứng dụng mà không cần định danh trực tiếp.

Trong mọi trường hợp, cơ chế tấn công là chung: *tính thừa + số chiều cao $\Rightarrow$ tính duy nhất $\Rightarrow$ khả năng liên kết*. Vì vậy, việc nghiên cứu kỹ tấn công Scoreboard-RH trên một ví dụ trực quan (phim) cung cấp khung tư duy để hiểu và phòng vệ trước cả lớp rủi ro giải ẩn danh trên thiết bị di động.

CHƯƠNG 2

PHÂN TÍCH BÀI TOÁN VÀ THIẾT KẾ HỆ THỐNG

Chương này mô tả việc chuyển từ lý thuyết sang một hệ thống mô phỏng chạy được: lựa chọn dữ liệu, kiến trúc phần mềm, mô hình dữ liệu và luồng tấn công.

2.1. Lựa chọn tập dữ liệu

Tập Netflix Prize gốc hiện không còn được phân phối công khai (chính vì các quan ngại riêng tư nói trên). Vì vậy chúng em dùng tập MovieLens 100k [3] làm proxy - một tập dữ liệu công khai do nhóm GroupLens (Đại học Minnesota) phát hành, có cùng bản chất (đánh giá phim kèm thời điểm) và cùng tính thưa. Thông số tập dữ liệu được tóm tắt ở Bảng 2.1.

Bảng 2.1. Thông số tập dữ liệu MovieLens 100k

Thuộc tính	Giá trị
Số người dùng	943
Số phim	1.682
Số lượt đánh giá	100.000
Thang điểm	1 - 5 (số nguyên)
Khoảng thời gian	09/1997 - 04/1998
Độ thưa (sparsity)	93,70%

Độ thưa 93,70% nghĩa là trong ma trận 943×1682 (người $\times$ phim), chỉ khoảng 6,3% ô có dữ liệu - phần còn lại để trống. Chính sự trống trải này tạo ra các tổ hợp phim gần như duy nhất cho mỗi người, đúng tiền đề mà thuật toán khai thác. Bảng 2.2 đối chiếu vai trò giữa hai tập dữ liệu.

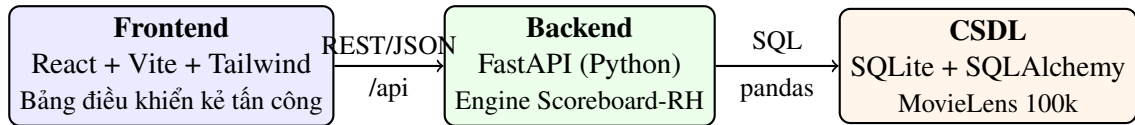
Do MovieLens nhỏ và dày hơn Netflix, các con số tuyệt đối sẽ khác đôi chút (đám đông ứng viên ít hơn), nhưng *xu hướng* - vốn là điều bài báo khẳng định - được tái hiện trung thực.

2.2. Kiến trúc hệ thống

Hệ thống mô phỏng theo mô hình client-server ba lớp, minh họa ở Hình 2.1.

Bảng 2.2. Đối chiếu tập Netflix gốc và proxy MovieLens 100k

Khái niệm	Netflix Prize	MovieLens 100k
Bản ghi một cá nhân	Một thuê bao	Một người dùng
Thuộc tính	(phim, điểm, ngày)	(phim, điểm, ngày)
Quy mô	~500.000 người	943 người
Tính thưa	Áp đảo	93,70%



Hình 2.1. Kiến trúc ba lớp của hệ thống mô phỏng tấn công giải ẩn danh

- **Lớp dữ liệu (SQLite):** lưu ba bảng `users`, `movies`, `ratings` được nạp từ tệp gốc MovieLens. Truy cập qua SQLAlchemy.
- **Lớp backend (FastAPI):** chứa *engine tấn công* - cài đặt thuật toán Scoreboard-RH bằng pandas/numpy. Toàn bộ 100.000 lượt đánh giá được nạp sẵn vào RAM khi khởi động và trọng số $wt(i)$ của mỗi phim được tính trước, nhờ đó mỗi lần tấn công chỉ mất vài chục mili-giây. Backend phơi bày REST API (xem Bảng 2.3).
- **Lớp frontend (React):** “bảng điều khiển kẻ tấn công” cho phép nhập tri thức hỗ trợ và trực quan hóa quá trình lộ lọt thông tin.

Bảng 2.3. Các điểm cuối (endpoint) REST của backend

Endpoint	Method	Chức năng
/stats	GET	Thống kê tập dữ liệu: độ thưa, phân phối điểm.
/movies/search	GET	Tìm phim theo tên, kèm support và trọng số $wt(i)$.
/attack	POST	Chạy thuật toán giải ẩn danh trên aux đã nhập.
/auto_aux	GET	Sinh aux từ một nạn nhân có thật (có thể thêm nhiều).

2.3. Mô hình dữ liệu

Ba bảng quan hệ được thiết kế tối giản, đủ cho bài toán:

- Bảng `users` gồm các cột: `id`, `age`, `gender`, `occupation`, `zip_code` - hồ sơ người dùng (đóng vai “thông tin nhạy cảm” có thể bị suy ra).

- Bảng `movies` gồm các cột: `id`, `title`, `release_date` - danh mục phim.
- Bảng `ratings` gồm các cột: `user_id`, `movie_id`, `rating`, `timestamp` - bảng trung tâm, 100.000 dòng, là đối tượng chính của tấn công.

2.4. Luồng tấn công

Một phiên tấn công đi qua các bước sau (tương ứng với thao tác trên giao diện ở Chương 4):

1. Kẻ tấn công thu thập tri thức bổ trợ: một vài phim mà nạn nhân đã xem, kèm điểm và (tùy chọn) ngày xem gần đúng.
2. Backend tính điểm cho mọi người dùng có chứa ít nhất một phim trong aux, theo công thức (1.3).
3. Backend tính eccentricity theo (1.4) và so với ngưỡng ϕ .
4. Nếu vượt ngưỡng: kết luận danh tính, đồng thời truy xuất và phơi bày toàn bộ lịch sử đánh giá của nạn nhân - phần dữ liệu mà kẻ tấn công ban đầu không hề biết.

Bước 4 chính là minh chứng trực quan cho “rò rỉ riêng tư”: từ vài mẫu thông tin vụn vặt, kẻ tấn công thu được toàn bộ chân dung hành vi của nạn nhân.

CHƯƠNG 3

CÀI ĐẶT THUẬT TOÁN VÀ NÂNG CẤP ỨNG DỤNG

Chương này trình bày chi tiết cách cài đặt thuật toán Scoreboard-RH và những tối ưu, nâng cấp chúng em thực hiện so với bản demo ban đầu. Mã nguồn đầy đủ của engine tấn công được in ở Phụ lục (Mã nguồn A.1).

3.1. Cài đặt hàm cho điểm

Trái tim của hệ thống là hàm `run_attack` trong lớp `AttackEngine`. Để đạt tốc độ, toàn bộ phép tính được vector hóa bằng `pandas/numpy` thay vì lặp trên từng người dùng. Quy trình:

1. Lọc ra mọi lượt đánh giá liên quan tới các phim trong `aux` (dùng `ratings_df.isin(target)`).
2. Với mỗi phim i trong `aux`: tính vector độ lệch điểm $|\rho_i - \rho'_i|$ cho tất cả người từng xem phim đó, đưa qua hàm mũ suy giảm, nhân trọng số $wt(i)$, rồi cộng dồn vào điểm của từng người.
3. Chuẩn hóa điểm về $[0, 1]$, sắp xếp giảm dần, tính eccentricity và đôi chiếu ngưỡng.

Đoạn lõi tính điểm (trích từ Mã nguồn A.1):

```
# Thanh phan rating: exp(-|rho - rho'| / rho_0)
r_diff = np.abs(m_ratings["rating"] - r_v)
sim = np.exp(-r_diff / rho_0)
denom += wt
# Thanh phan thoi gian (cong vao, dung cong thuc paper)
if use_dates and t_v is not None:
    t_diff = np.abs(m_ratings["timestamp"] - t_v)
    sim = sim + np.exp(-t_diff / d_0)
    denom += wt
temp = pd.Series((wt * sim).values, index=m_ratings["user_id"])
user_scores = user_scores.add(temp, fill_value=0)
```

Trọng số $wt(i)$ được tính sẵn một lần khi nạp dữ liệu, dùng $\log(\text{supp} + 1)$ để ổn định số học (tránh chia cho $\log 1 = 0$ với phim chỉ một người xem):

```

supp = self.ratings_df.groupby("movie_id").size()
self.movie_weights = 1.0 / np.log(supp + 1)

```

3.2. Cài đặt tiêu chí eccentricity

Sau khi có điểm chuẩn hóa, eccentricity được tính trực tiếp theo công thức (1.4):

```

sorted_scores = norm_scores.sort_values(ascending=False)
max_score = sorted_scores.iloc[0]
max2_score = sorted_scores.iloc[1]
sigma = sorted_scores.std()
eccentricity = (max_score - max2_score) / sigma if sigma > 0 else 0.0
if eccentricity > phi:
    # ... ket luan danh tinh, truy xuat toan bo lich su nan nhan

```

Chỉ khi vượt ngưỡng ϕ , hệ thống mới truy vấn JOIN để lấy toàn bộ lịch sử của người bị định danh - đây là dữ liệu “bị phơi bày”.

3.3. Các nâng cấp so với bản demo ban đầu

Bản demo được cung cấp ban đầu chạy được nhưng có một số điểm chưa trung thực với bài báo và còn thiếu tính năng. Chúng em đã thực hiện các nâng cấp sau:

3.3.1. Sửa hàm cho điểm cho đúng công thức gốc

Bản demo ban đầu nhân hai thành phần tương tự ($\text{sim}_r \times \text{sim}_t$). Cách này khiến điểm sụp về 0 khi chỉ một thành phần lệch, không khớp với công thức (1.3) của bài báo vốn cộng hai thành phần. Chúng em đã sửa lại thành phép cộng đúng như (1.3). Khác biệt này quan trọng: phép cộng cho phép thuật toán vẫn ghi nhận một phim ngay cả khi ngày xem sai lệch nhiều - đúng tinh thần “bền vững” (robust) trong tên thuật toán.

3.3.2. Bổ sung chế độ “không dùng ngày”

Chúng em thêm tham số `use_dates`: khi tắt, hàm cho điểm bỏ hẳn thành phần thời gian, mô phỏng kịch bản kẻ tấn công chỉ biết *phim* và *điểm* (tái hiện Fig. 8 của bài báo). Nhờ đó báo cáo có thể đo riêng đóng góp của thông tin ngày (Chương 4).

3.3.3. Phơi bày trọng số độ hiếm trên giao diện

Endpoint `/movies/search` và `/auto_aux` nay trả về kèm `support` (số người xem) và `weight` ($wt(i)$) của từng phim. Giao diện hiển thị trực tiếp các giá trị này (Hình 4.2), giúp người xem hiểu vì sao phim hiếm lại “đắt giá” cho việc định danh.

3.3.4. Sinh nạn nhân thật và mô phỏng nhiễu

Chúng em bổ sung hàm `sample_aux_from_user` (trong Mã nguồn A.1) cho phép lấy “dấu vân tay” của một người dùng có thật trong CSDL, rồi cố ý thêm nhiễu - sai số điểm ± 1 và sai số ngày ± 14 ngày - trước khi tấn công. Tính năng này phục vụ hai mục đích: (1) tạo kịch bản demo lặp lại được; (2) chứng minh trực tiếp tính bền vững của thuật toán (Định lý 1 của bài báo). Khi sinh aux có thể ưu tiên phim hiếm (`prefer_rare`) để thấy rõ sức mạnh của các thuộc tính hiếm.

3.3.5. Module thực nghiệm định lượng

Cuối cùng, chúng em viết `experiment.py` (Mã nguồn A.3) để chạy tấn công hàng loạt trên hàng trăm nạn nhân, đo *tỉ lệ định danh đúng* và *eccentricity trung bình* theo số lượng phim biết trước, trong bốn kịch bản khác nhau. Đây là cơ sở cho các biểu đồ định lượng ở Chương 4. Một lần định danh được tính là thành công chỉ khi $ecc > \phi$ và người được chọn đúng là nạn nhân thật (true positive).

3.4. Công nghệ và môi trường

Backend dùng Python 3.12, FastAPI [14], pandas, numpy, SQLAlchemy; frontend dùng React 18 [15], Vite, Tailwind CSS, Recharts. Vite proxy đường dẫn `/api` sang backend ở cổng 8000, tránh vấn đề CORS khi phát triển. Hướng dẫn cài đặt và chạy đầy đủ nằm ở Phụ lục.

CHƯƠNG 4

THỰC NGHIỆM VÀ KẾT QUẢ

Chương này trình bày kết quả thật thu được từ hệ thống: trước hết là một phiên tấn công minh họa qua giao diện, sau đó là các kết quả định lượng trên hàng trăm nạn nhân. Mọi ảnh chụp và số liệu đều sinh trực tiếp từ ứng dụng.

4.1. Môi trường thực nghiệm

Thực nghiệm chạy trên máy cá nhân, hệ điều hành Linux, Python 3.12 và Node.js 24. Backend nạp 100.000 lượt đánh giá vào RAM trong khoảng 1 giây; mỗi lần tấn công hoàn tất dưới 50 mili-giây. Tập dữ liệu là MovieLens 100k công khai như mô tả ở Chương 2.

4.2. Một phiên tấn công qua giao diện

4.2.1. Tổng quan hệ thống

Hình 4.1 là màn hình chính. Bảng *Dataset Sparsity* hiển thị độ thưa thớt 93,6953% tính trực tiếp từ CSDL, cùng phân phối điểm. Bên phải là khu vực trực quan hóa quá trình tấn công, ban đầu để trống.

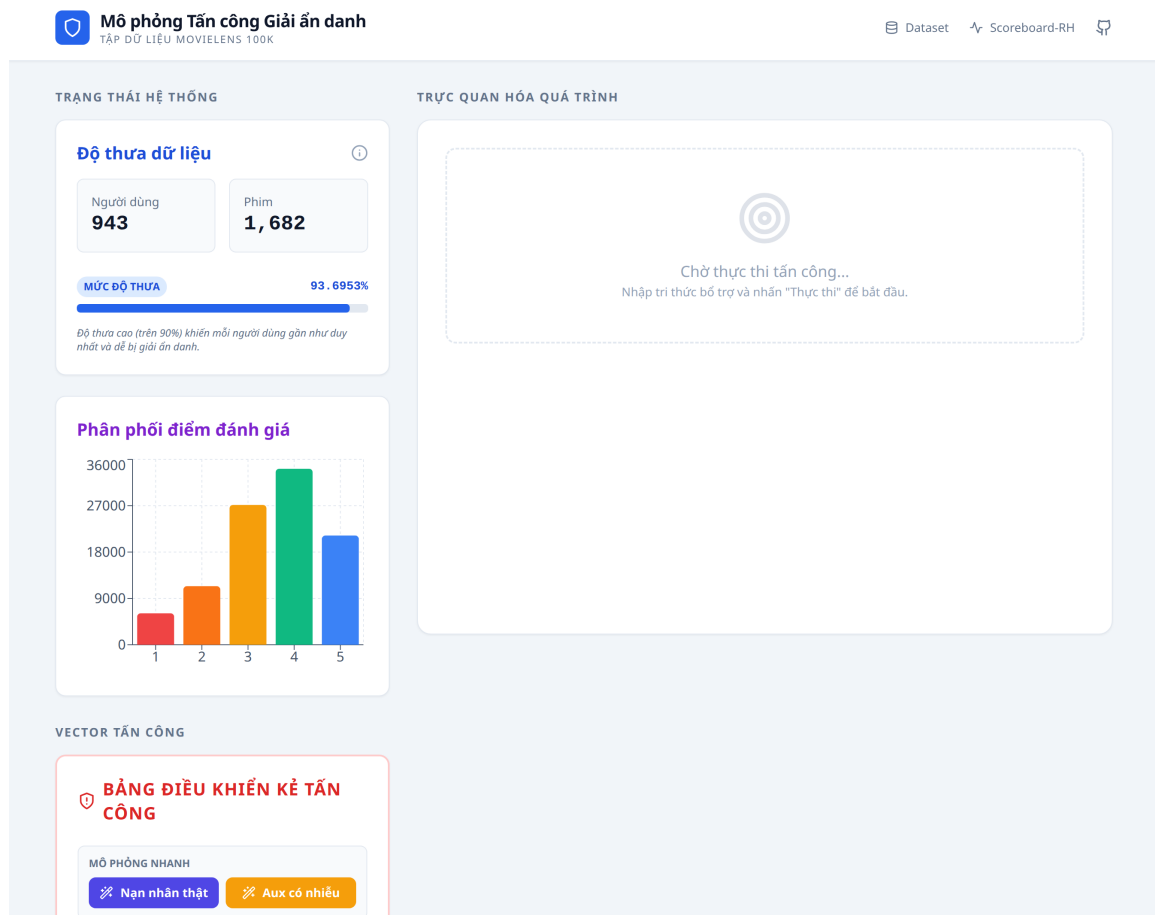
4.2.2. Nạp tri thức hỗ trợ

Chúng em dùng nút “Nạn nhân thật” để hệ thống lấy ngẫu nhiên dấu vân tay của một người dùng có thật. Hình 4.2 cho thấy bốn phim được nạp vào bảng *Thông tin đã biết*, mỗi phim kèm điểm, ngày, số người xem (support) và trọng số $wt(i)$. Có thể thấy các phim ít người xem (support nhỏ) được gán trọng số lớn hơn - đúng công thức (1.2).

4.2.3. Kết quả định danh

Sau khi nhấn “Thực thi giải ẩn danh”, thuật toán xác định chính xác nạn nhân (Hình 4.3): User #373, eccentricity 3,8333 (vượt xa ngưỡng 1,5), độ tương tự 0,9886. Đáng chú ý, không gian ứng viên chỉ còn 24 người - vì bốn phim hiếm trong aux đã thu hẹp đám đông xuống rất nhanh.

Quan trọng hơn, hệ thống phơi bày toàn bộ lịch sử xem phim của người này trong bảng *Lịch sử riêng tư bị phơi bày* (Hình 4.4) - 233 lượt đánh giá mà kẻ tấn công ban đầu



Hình 4.1. Màn hình chính: thống kê độ thừa và bảng điều khiển kẻ tấn công

hoàn toàn không biết. Đây chính là biểu hiện trực quan của “rò rỉ riêng tư”: từ 4 mẫu thông tin, kẻ tấn công thu được toàn bộ chân dung hành vi của nạn nhân.

Hình 4.5 là bảng xếp hạng *Các ứng viên hàng đầu*: ứng viên hạng nhất (độ tương tự $\approx 0,99$) bỏ xa các ứng viên còn lại - minh họa trực quan ý nghĩa của eccentricity và lý do thuật toán tự tin kết luận.

4.2.4. Tính bền vững trước thông tin nhiễu

Để kiểm chứng tính bền vững, chúng em dùng nút “Aux có nhiều”: hệ thống lấy 6 phim của một nạn nhân thật nhưng cố ý làm sai điểm ± 1 và sai ngày ± 14 ngày. Dù vậy, thuật toán vẫn định danh đúng (Hình 4.6): User #278, eccentricity 5,1695. Điều này xác nhận đặc tính “robust” cốt lõi: tấn công không đòi hỏi tri thức bổ trợ chính xác.

4.3. Kiểm chứng trên dòng lệnh

Để minh bạch, chúng em cũng chạy trực tiếp engine trên dòng lệnh. Hình 4.7 cho thấy một phiên tấn công vào nạn nhân thật #200 với 6 phim chính xác: trong số 782 ứng

Đã định danh người dùng!

ĐIỂM ECCENTRICITY
3.8333

24 ứng viên · chế độ có ngày

ID người dùng mục tiêu
#373

Độ tương tự
0.988575

Độ tin cậy
Cao

Hình 4.3. Định danh thành công User #373 với eccentricity 3,8333

Lịch sử riêng tư bị phơi bày (233 lượt xem)

Tên phim	Điểm	Ngày
That Darn Cat! (1965)	★ 3	10/18/1997
Big Squeeze, The (1996)	★ 3	10/18/1997
Escape to Witch Mountain (1975)	★ 3	10/18/1997
Star Trek: Generations (1994)	★ 4	10/18/1997
Alaska (1996)	★ 3	10/18/1997
Dragonheart (1996)	★ 3	10/18/1997
Another Stakeout (1993)	★ 1	10/18/1997
Pretty Woman (1990)	★ 3	10/18/1997
Bed of Roses (1996)	★ 5	10/18/1997

Hình 4.4. Lịch sử riêng tư của nạn nhân bị phơi bày hoàn toàn (233 lượt xem)

Hình 4.8 và tổng hợp ở Bảng 4.1.

Bảng 4.1. Tỷ lệ định danh đúng (%) theo số phim biết trước và kịch bản

Kịch bản	2 phim	3 phim	4 phim	6 phim	8 phim
Chính xác + có ngày	39,0	71,3	84,3	94,3	98,0
Chính xác, không ngày	10,7	30,7	55,7	83,7	93,0
Nhiều (± 1 điểm, ± 14 ngày)	12,7	29,0	43,3	60,3	72,3
Ưu tiên phim hiếm, không ngày	58,7	85,3	93,7	98,0	99,7

👤 Các ứng viên hàng đầu		
HẠNG 1	User #373	Độ tương tự 0.988575
HẠNG 2	User #181	Độ tương tự 0.267344
HẠNG 3	User #279	Độ tương tự 0.219838
HẠNG 4	User #389	Độ tương tự 0.181445
HẠNG 5	User #222	Độ tương tự 0.170300
HẠNG 6	User #145	Độ tương tự 0.155771
HẠNG 7	User #7	Độ tương tự 0.154910
HẠNG 8	User #846	Độ tương tự 0.148255
HẠNG 9	User #303	Độ tương tự 0.143063
HẠNG 10	User #116	Độ tương tự 0.135572

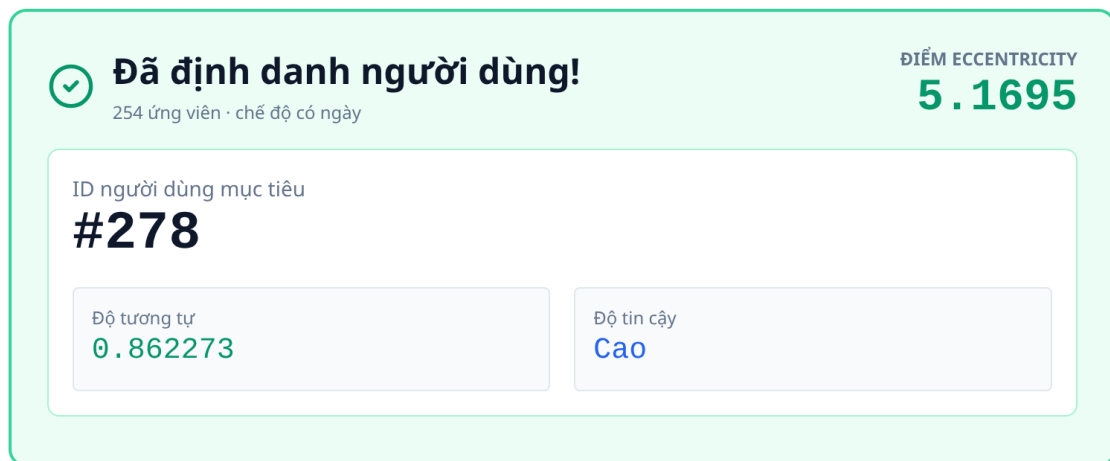
Hình 4.5. Bảng xếp hạng ứng viên: hạng nhất nổi bật hẳn so với phần còn lại

Hình 4.9 và 4.10 trực quan hóa Bảng 4.1 và eccentricity trung bình tương ứng.

4.5. Phân tích kết quả

Các con số thật trên MovieLens tái hiện trung thực những kết luận của bài báo gốc:

- **Càng nhiều tri thức, càng dễ định danh.** Với 8 phim chính xác kèm ngày, tỉ lệ định danh đúng đạt 98% - rất gần con số 99% mà bài báo công bố trên Netflix.
- **Ngày xem là thông tin giá trị nhưng không bắt buộc.** Bỏ thông tin ngày làm tỉ lệ giảm (ví dụ với 4 phim: từ 84,3% xuống 55,7%), nhưng khi biết đủ 8 phim vẫn đạt 93%. Quyền riêng tư không được cứu vãn chỉ bằng cách giấu ngày.



Hình 4.6. Vẫn định danh đúng dù aux có nhiễu (± 1 điểm, ± 14 ngày)

```
son@Fremontz:~/ANDD/backend$ source venv/bin/activate && python3 demo_attack.py
Loading ratings data into memory...
Loaded 100000 ratings for 1682 movies.

[+] Nan nhan that: user #200
[+] Tri thuc bo tro (aux) ke tan cong nam giu (6 phim, chinh xac):
  - Tombstone (1993)          rating=4 support=108 wt=0.213
  - Contact (1997)           rating=4 support=509 wt=0.160
  - Saint, The (1997)         rating=3 support=316 wt=0.174
  - Star Trek: The Motion Picture (1979) rating=5 support=117 wt=0.210
  - Fugitive, The (1993)      rating=5 support=336 wt=0.172
  - Birdcage, The (1996)      rating=4 support=293 wt=0.176

[+] Tong so ung vien xet den : 782
[+] Eccentricity : 4.2504 (nguong phi=1.5)
[+] Top-3 ung vien :
  #1 user 200 similarity=1.0000
  #2 user 846 similarity=0.5582
  #3 user 899 similarity=0.5518

[!] DINH DANH THANH CONG -> user #200 (DUNG nan nhan)
[!] Da phoi bay 216 luot xem phim riêng tu cua nan nhan.

son@Fremontz:~/ANDD/backend$
```

Hình 4.7. Một phiên tấn công thật chạy trên dòng lệnh (nạn nhân #200)

- **Tấn công bền vững trước nhiễu.** Ngay cả khi điểm sai ± 1 và ngày sai ± 14 ngày, 8 phim vẫn cho 72,3%. Tri thức bổ trợ *không cần chính xác*.
- **Phim hiếm là vũ khí mạnh nhất.** Khi ưu tiên phim ít người xem, chỉ 2 phim đã đủ định danh 58,7% - cao hơn cả việc biết 4 phim phổ biến. Điều này khẳng định vai trò của trọng số độ hiếm $wt(i)$: một bộ phim nghệ thuật ít người biết tiết lộ nhiều hơn nhiều so với một bom tấn.
- **Eccentricity tăng đều theo lượng tri thức** (Hình 4.10), cho thấy ứng viên đúng ngày càng “nổi bật” và cơ chế tự kiểm soát sai số hoạt động ổn định.

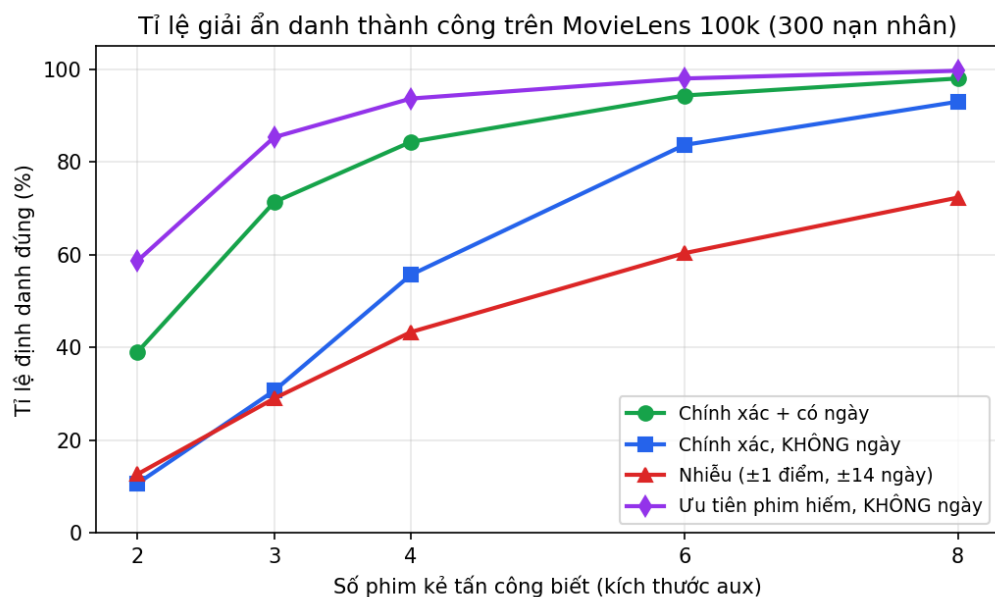
```

song@Fremontz:~/ANDD/backend$ source venv/bin/activate && python3 experiment.py
Loading ratings data into memory...
Loaded 100000 ratings for 1682 movies.
scenario      k      rate    mean ecc      n
exact_dates   2    0.390    1.504    300
exact_dates   3    0.713    2.558    300
exact_dates   4    0.843    3.379    300
exact_dates   6    0.943    4.322    300
exact_dates   8    0.980    4.877    300
exact_nodates  2    0.107    0.403    300
exact_nodates  3    0.307    1.160    300
exact_nodates  4    0.557    1.848    300
exact_nodates  6    0.837    2.851    300
exact_nodates  8    0.930    3.424    300
noisy_dates   2    0.127    0.685    300
noisy_dates   3    0.290    1.209    300
noisy_dates   4    0.433    1.697    300
noisy_dates   6    0.603    2.419    300
noisy_dates   8    0.723    2.845    300
rare_nodates  2    0.587    2.284    300
rare_nodates  3    0.853    2.480    300
rare_nodates  4    0.937    3.251    300
rare_nodates  6    0.980    4.376    300
rare_nodates  8    0.997    5.207    300

Đã ghi 20 dòng vào /home/son/KTANM_final_report/ANDD/backend/results/deanon_results.csv
song@Fremontz:~/ANDD/backend$

```

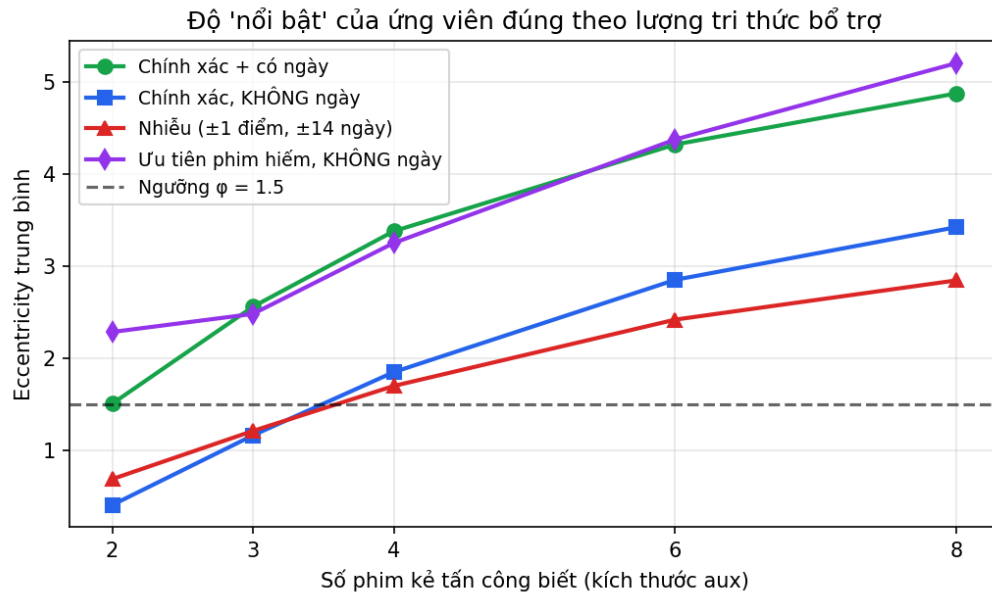
Hình 4.8. Kết xuất (log) phiên chạy thực nghiệm trên 300 nạn nhân tại giao diện dòng lệnh, 4 kịch bản $\times$ 5 mức số phim



Hình 4.9. Tỉ lệ định danh thành công theo lượng tri thức bổ trợ

4.6. Tình huống thất bại và ý nghĩa của ngưỡng

Không phải mọi cuộc tấn công đều thành công - và đó là chủ ý thiết kế. Khi tri thức bổ trợ ít hoặc chỉ gồm phim phổ biến, nhiều người dùng có hồ sơ tương tự nhau, eccentricity tụt xuống dưới 1,5 và thuật toán từ chối định danh thay vì đoán bừa. Trong một thực nghiệm với 5 phim phổ biến kèm nhiều, ứng viên hạng nhất tuy đúng là nạn nhân nhưng eccentricity chỉ đạt 1,4995 - sát ngưỡng - nên hệ thống vẫn báo “không đủ tin cậy”. Cơ chế ngưỡng này giữ tỉ lệ dương tính giả ở mức thấp, khiến mỗi lần “định



Hình 4.10. Eccentricity trung bình theo lượng tri thức bổ trợ (đường nét đứt là ngưỡng $\phi = 1,5$)

đanh thành công” đều có độ tin cậy cao - đây chính là điều làm cho tấn công trở nên *nguy hiểm* trên thực tế.

CHƯƠNG 5

THẢO LUẬN, PHÒNG VỆ VÀ KẾT LUẬN

5.1. Ý nghĩa về quyền riêng tư

Thực nghiệm cho thấy một thông điệp rõ ràng: ẩn danh hóa không đồng nghĩa với riêng tư. Việc gỡ bỏ tên và số định danh khỏi một tập micro-data thừa gần như không bảo vệ được người dùng, vì chính tổ hợp hành vi của họ đã là một “dấu vân tay” duy nhất. Hệ quả còn nghiêm trọng hơn với đặc tính forward privacy: một khi một mẫu dữ liệu bị liên kết với danh tính thật, mọi liên kết về sau (kể cả với các định danh ảo mới) đều trở nên dễ dàng hơn. Trong bối cảnh thiết bị và ứng dụng di động liên tục thu thập dữ liệu vị trí, lịch sử duyệt web, danh sách ứng dụng đã cài - vốn cũng thừa và nhiều chiều - rủi ro giải ẩn danh là hoàn toàn hiện hữu.

5.2. Các biện pháp phòng vệ

5.2.1. Vì sao *k-anonymity* không đủ

Như đã phân tích ở Chương 1, *k-anonymity* giả định tách được quasi-identifier khỏi thuộc tính nhạy cảm - điều bất khả thi với dữ liệu hành vi, nơi mọi thuộc tính đều có thể là điểm liên kết. Trên dữ liệu nhiều chiều, để đạt *k*-ẩn danh phải khái quát hóa tới mức dữ liệu mất giá trị sử dụng.

5.2.2. *Riêng tư vi phân (Differential Privacy)*

Hướng phòng vệ vững chắc nhất hiện nay là riêng tư vi phân [12]: thay vì công bố micro-data, hệ thống chỉ trả lời các truy vấn tổng hợp và cố ý thêm nhiễu được hiệu chỉnh toán học, sao cho sự hiện diện hay vắng mặt của *một* cá nhân gần như không ảnh hưởng tới kết quả. Cách này cung cấp bảo đảm riêng tư có thể chứng minh được, đánh đổi bằng một phần độ chính xác.

5.2.3. Các biện pháp thực tiễn khác

- **Không công bố micro-data ở mức cá nhân**; chỉ phát hành thống kê tổng hợp đã thêm nhiễu.
- **Thêm nhiễu mạnh và lấy mẫu nhỏ**: tuy nhiên bài báo chứng minh nhiễu nhẹ là vô tác dụng - phải đủ mạnh tới mức ảnh hưởng giá trị sử dụng mới có hiệu quả.

- **Giảm chiều và gộp nhóm thuộc tính:** loại bỏ hoặc gộp các thuộc tính hiếm (vốn mang giá trị định danh cao nhất) trước khi công bố.
- **Kiểm soát truy cập và ràng buộc pháp lý:** hợp đồng sử dụng dữ liệu, cấm tái định danh, tối thiểu hóa dữ liệu thu thập ngay từ đầu.

5.3. Hạn chế của nghiên cứu

- Tập MovieLens 100k nhỏ hơn nhiều so với Netflix (943 so với 500.000 người), nên đám đông ứng viên ít hơn và một số con số tuyệt đối lạc quan hơn thực tế; tuy nhiên các *xu hướng* được tái hiện trung thực.
- Các tham số ρ_0, d_0, ϕ lấy theo bài báo, chưa tinh chỉnh riêng cho MovieLens.
- Hệ thống mô phỏng kịch bản kẻ tấn công đã biết bản ghi nạn nhân nằm trong tập công bố; chưa cài đặt đầy đủ biến thể phát hiện “nạn nhân không có trong mẫu”.

5.4. Kết luận

Báo cáo đã nghiên cứu, mô phỏng và kiểm chứng bằng thực nghiệm thật lớp tấn công giải ẩn danh trên dữ liệu thừa của Narayanan-Shmatikov. Chúng em đã: (1) trình bày lại nền tảng lý thuyết và thuật toán Scoreboard-RH; (2) xây dựng, sửa lỗi và nâng cấp một ứng dụng web hoàn chỉnh - đặc biệt là sửa hàm cho điểm cho đúng công thức gốc, bổ sung chế độ không-ngày, phơi bày trọng số độ hiếm và mô-đun thực nghiệm; (3) tái hiện thành công các kết quả chính của bài báo trên MovieLens 100k, với 8 phim cho tỉ lệ định danh đúng tới 98%, và chứng minh tính bền vững trước nhiễu cũng như sức mạnh đặc biệt của các thuộc tính hiếm.

Kết quả khẳng định một bài học nền tảng của an toàn dữ liệu hiện đại: trong thế giới dữ liệu thừa và nhiều chiều, riêng tư phải được bảo đảm bằng các cơ chế có cơ sở toán học như riêng tư vi phân, chứ không thể trông cậy vào việc “gỡ bỏ thông tin định danh”.

TÀI LIỆU THAM KHẢO

- [1] A. Narayanan, V. Shmatikov, “Robust De-anonymization of Large Sparse Datasets,” *Proc. IEEE Symposium on Security and Privacy (S&P)*, pp. 111-125, 2008. DOI: <https://doi.org/10.1109/SP.2008.33>
- [2] J. Bennett, S. Lanning, “The Netflix Prize,” *Proc. KDD Cup and Workshop*, 2007. [Online]. Available: <https://web.archive.org/web/20090924184639/http://www.netflixprize.com/>
- [3] F. M. Harper, J. A. Konstan, “The MovieLens Datasets: History and Context,” *ACM Transactions on Interactive Intelligent Systems (TiiS)*, vol. 5, no. 4, art. 19, 2015. DOI: <https://doi.org/10.1145/2827872>
- [4] L. Sweeney, “k-anonymity: A Model for Protecting Privacy,” *International Journal of Uncertainty, Fuzziness and Knowledge-Based Systems*, vol. 10, no. 5, pp. 557-570, 2002. DOI: <https://doi.org/10.1142/S0218488502001648>
- [5] A. Machanavajjhala, J. Gehrke, D. Kifer, M. Venkitasubramanian, “l-diversity: Privacy Beyond k-anonymity,” *Proc. IEEE ICDE*, 2006. DOI: <https://doi.org/10.1109/ICDE.2006.1>
- [6] N. Li, T. Li, S. Venkatasubramanian, “t-Closeness: Privacy Beyond k-Anonymity and l-Diversity,” *Proc. IEEE ICDE*, pp. 106-115, 2007. DOI: <https://doi.org/10.1109/ICDE.2007.367856>
- [7] M. Barbaro, T. Zeller, “A Face Is Exposed for AOL Searcher No. 4417749,” *The New York Times*, Aug. 2006. [Online]. Available: <https://www.nytimes.com/2006/08/09/technology/09aol.html>
- [8] Y.-A. de Montjoye, C. A. Hidalgo, M. Verleysen, V. D. Blondel, “Unique in the Crowd: The Privacy Bounds of Human Mobility,” *Scientific Reports*, vol. 3, art. 1376, 2013. DOI: <https://doi.org/10.1038/srep01376>

- [9] Y.-A. de Montjoye, L. Radaelli, V. K. Singh, A. “Sandy” Pentland, “Unique in the Shopping Mall: On the Reidentifiability of Credit Card Metadata,” *Science*, vol. 347, no. 6221, pp. 536-539, 2015. DOI: <https://doi.org/10.1126/science.1256297>
- [10] Electronic Privacy Information Center, “The Video Privacy Protection Act (VPPA).” [Online]. Available: <https://epic.org/privacy/vppa/>
- [11] D. Frankowski, D. Cosley, S. Sen, L. Terveen, J. Riedl, “You Are What You Say: Privacy Risks of Public Mentions,” *Proc. ACM SIGIR*, pp. 565-572, 2006. DOI: <https://doi.org/10.1145/1148170.1148267>
- [12] C. Dwork, “Differential Privacy,” *Proc. ICALP 2006*, LNCS 4052, pp. 1-12, 2006. DOI: https://doi.org/10.1007/11787006_1
- [13] R. Singel, “Netflix Spilled Your Brokeback Mountain Secret, Lawsuit Claims,” *Wired*, Dec. 2009. [Online]. Available: <https://www.wired.com/2009/12/netflix-privacy-lawsuit/>
- [14] S. Ramírez, “FastAPI - Modern, Fast Web Framework for Building APIs with Python.” [Online]. Available: <https://fastapi.tiangolo.com/>
- [15] Meta Open Source, “React - The Library for Web and Native User Interfaces.” [Online]. Available: <https://react.dev/>

PHỤ LỤC

A. Hướng dẫn cài đặt và chạy demo

Mã nguồn dự án gồm hai phần: backend/ (FastAPI + engine tấn công) và frontend/ (React + Vite). Các bước chạy:

Backend (cổng 8000):

```
cd backend
python3 -m venv venv && source venv/bin/activate
pip install -r requirements.txt
python data_loader.py          # nạp MovieLens 100k vào SQLite (chỉ lan đầu)
uvicorn main:app --reload      # chạy API tại http://localhost:8000
```

Frontend (cổng 5173):

```
cd frontend
npm install
npm run dev                # mở http://localhost:5173
```

Chạy thực nghiệm định lượng và vẽ biểu đồ:

```
cd backend && source venv/bin/activate
python experiment.py          # sinh results/deanon_results.csv
python plot_results.py        # sinh success_rate.png, eccentricity.png
python demo_attack.py         # một phiên tấn công mẫu trên dòng lệnh
```

B. Mã nguồn các thành phần chính

Phần này in mã nguồn đầy đủ của các thành phần cốt lõi, được tham chiếu trực tiếp từ Chương 3.

Mã nguồn A.1. `attack_engine.py` - engine cài đặt thuật toán Scoreboard-RH, gồm hàm cho điểm, `eccentricity` và `sample_aux_from_user`

```
import pandas as pd
import numpy as np
from database import get_db_engine

class AttackEngine:
    """
    Cài đặt thuật toán giải an danh Scoreboard-RH của Narayanan & Shmatikov
```

("Robust De-anonymization of Large Sparse Datasets", 2008) tren tap MovieLens 100k.

Ham cho diem theo dung cong thuc trong bai bao (muc 5):

$$\text{Score}(\text{aux}, r') = \sum_{i \in \text{supp}(\text{aux})} \text{wt}(i) * (\exp(-|\rho_i - \rho'_i| / \rho_0) + \exp(-|d_i - d'_i| / d_0))$$

voi trong so $\text{wt}(i) = 1 / \log(|\text{supp}(i)|)$ (phim càng ít người xem càng nặng).
Tieu chi nhan dang (eccentricity):

$\text{ecc} = (\text{max1} - \text{max2}) / \text{sigma} > \text{phi}$ thi ket luan dinh danh duoc.
"""

```
def __init__(self):
    self.engine = get_db_engine()
    self.ratings_df = None
    self.movie_weights = None
    self.load_data()

def load_data(self):
    """Nap toan bo ratings vao RAM va tinh san trong so moi phim."""
    print("Loading ratings data into memory...")
    self.ratings_df = pd.read_sql(
        "SELECT user_id, movie_id, rating, timestamp FROM ratings", self.engine
    )

    # wt(i) = 1 / log(|supp(i)|); supp(i) = so nguoi da xem phim i.
    # Dung log(supp + 1) de on dinh so hoc (tranh chia cho log(1) = 0).
    supp = self.ratings_df.groupby("movie_id").size()
    self.movie_supp = supp
    self.movie_weights = 1.0 / np.log(supp + 1)
    print(f"Loaded {len(self.ratings_df)} ratings for {len(supp)} movies.")

def get_movie_weight(self, movie_id):
    """Tra ve (support, weight) cua mot phim de hien thi tren giao dien."""
    s = int(self.movie_supp.get(movie_id, 0))
    w = float(self.movie_weights.get(movie_id, 0.0))
    return s, w

def run_attack(self, known_ratings, rho_0=1.5, d_0=30 * 24 * 3600,
               phi=1.5, use_dates=True):
    """
    Thuc thi tan cong giai an danh.

    known_ratings: list dict [{'movie_id', 'rating', 'timestamp'}]
    rho_0: he so co gian ve rating (paper dung 1.5)
    d_0: he so co gian ve thoi gian, don vi giay (paper dung 30 ngay)
    phi: nguong eccentricity (paper dung 1.5)
    use_dates: neu False thi bo han thanh phan ngay (kich ban ke tan cong
               chi biet phim + diem, tai hien Fig.8 cua bai bao).
    """
    if not known_ratings:
```

```

        return {"error": "No known ratings provided"}

    target_movie_ids = [r["movie_id"] for r in known_ratings]
    relevant = self.ratings_df[
        self.ratings_df["movie_id"].isin(target_movie_ids)
    ].copy()

    if relevant.empty:
        return {"error": "No matching movies found in database"}

    unique_users = relevant["user_id"].unique()
    user_scores = pd.Series(0.0, index=unique_users)

    # Chuan hoa diem ve [0, 1] de hien thi: moi phim dong gop toi da
    # (2*wt neu tinh ngay, nguoc lai wt). Eccentricity khong phu thuoc
    # phep chuan hoa nay (no chia cho do lech chuan).
    denom = 0.0
    aux_weights = []

    for kr in known_ratings:
        m_id = kr["movie_id"]
        r_v = kr["rating"]
        t_v = kr.get("timestamp")
        wt = float(self.movie_weights.get(m_id, 0.0))
        aux_weights.append({"movie_id": m_id,
                           "support": int(self.movie_supp.get(m_id, 0)),
                           "weight": wt})

        if wt == 0:
            continue

        m_ratings = relevant[relevant["movie_id"] == m_id]
        if m_ratings.empty:
            continue

        # Thanh phan rating:  $\exp(-|rho - rho'| / rho_0)$ 
        r_diff = np.abs(m_ratings["rating"] - r_v)
        sim = np.exp(-r_diff / rho_0)
        denom += wt

        # Thanh phan thoi gian (cong vao, theo dung cong thuc paper)
        if use_dates and t_v is not None:
            t_diff = np.abs(m_ratings["timestamp"] - t_v)
            sim = sim + np.exp(-t_diff / d_0)
            denom += wt

        temp = pd.Series((wt * sim).values, index=m_ratings["user_id"])
        user_scores = user_scores.add(temp, fill_value=0)

    if user_scores.empty or denom == 0:
        return {"error": "No scores calculated"}

    # Chuan hoa hien thi
    norm_scores = user_scores / denom
    sorted_scores = norm_scores.sort_values(ascending=False)

```

```

max_score = sorted_scores.iloc[0]
if len(sorted_scores) > 1:
    max2_score = sorted_scores.iloc[1]
    sigma = sorted_scores.std()
    eccentricity = (max_score - max2_score) / sigma if sigma > 0 else 0.0
else:
    eccentricity = 100.0 # Chi co duy nhat mot ung vien

identified_user = None
full_history = []
if eccentricity > phi:
    u_id = int(sorted_scores.index[0])
    identified_user = {
        "user_id": u_id,
        "similarity": float(sorted_scores.iloc[0]),
    }
    history_query = """
        SELECT m.title, r.rating, r.timestamp
        FROM ratings r
        JOIN movies m ON r.movie_id = m.id
        WHERE r.user_id = ?
        ORDER BY r.timestamp DESC
    """
    history_df = pd.read_sql(history_query, self.engine, params=(u_id,))
    full_history = history_df.to_dict(orient="records")

return {
    "top_candidates": [
        {"user_id": int(u_id), "similarity": float(score)}
        for u_id, score in sorted_scores.head(10).items()
    ],
    "eccentricity": float(eccentricity),
    "identified_user": identified_user,
    "full_history": full_history,
    "total_candidates": int(len(sorted_scores)),
    "aux_weights": aux_weights,
    "use_dates": bool(use_dates),
}

# -----
# Tien ich phuc vu kich ban demo & thuc nghiem
# -----
def sample_aux_from_user(self, user_id, num_movies=3, rating_error=0,
                        date_error_days=0, seed=None, prefer_rare=False):
    """
    Sinh "tri thuc bo tro" (aux) tu mot nguoi dung co that trong CSDL,
    co the them nhieu (sai so rating, sai so ngay) de chung minh tinh
    BEN VUNG cua thuat toan truoc thong tin khong chinh xac ? dung tinh
    than Dinh ly 1 cua bai bao.
    """
    rng = np.random.RandomState(seed)
    urows = self.ratings_df[self.ratings_df["user_id"] == user_id]
    if urows.empty:
        return []
    if prefer_rare:

```

```

        # Ưu tiên phim hiếm (ít người xem) - mạnh hơn cho danh sách
        urows = urows.assign(
            _w=urows["movie_id"].map(self.movie_weights).fillna(0)
        ).sort_values("_w", ascending=False)
        picked = urows.head(num_movies)
    else:
        n = min(num_movies, len(urows))
        idx = rng.choice(len(urows), size=n, replace=False)
        picked = urows.iloc[idx]

    aux = []
    for row in picked.itertuples():
        rating = float(row.rating)
        if rating_error > 0:
            rating += rng.randint(-rating_error, rating_error + 1)
            rating = float(min(5, max(1, rating)))
        ts = int(row.timestamp)
        if date_error_days > 0:
            ts += int(rng.randint(-date_error_days, date_error_days + 1)) * 86400
        aux.append({"movie_id": int(row.movie_id),
                    "rating": rating, "timestamp": ts})

    return aux

```

Mã nguồn A.2. main.py - REST API FastAPI phơi bày các endpoint tấn công

```

from fastapi import FastAPI, Body, Query
from fastapi.middleware.cors import CORSMiddleware
from stats import get_sparsity_stats
from attack_engine import AttackEngine
from database import get_db_engine
from typing import Dict, Any
import pandas as pd

app = FastAPI(title="De-anonymization Demo API")

# CORS configuration
app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

# Initialize Attack Engine ( nạp dữ liệu một lần khi khởi động)
attack_engine = AttackEngine()

@app.get("/stats")
async def get_stats():
    """Tra về thống kê tập dữ liệu: độ thưa và phân phối điểm."""
    try:
        return get_sparsity_stats()
    except Exception as e:
        return {"error": str(e)}

```

```

@app.get("/movies/search")
async def search_movies(q: str = Query(..., min_length=1)):
    """Tim phim theo ten, kem support va trong so wt(i) cua tung phim."""
    try:
        engine = get_db_engine()
        query = "SELECT id, title, release_date FROM movies WHERE title LIKE ? LIMIT 10"
        movies = pd.read_sql(query, engine, params=(f"%{q}%",))
        records = movies.to_dict(orient="records")
        for r in records:
            s, w = attack_engine.get_movie_weight(r["id"])
            r["support"] = s
            r["weight"] = round(w, 4)
        return records
    except Exception as e:
        return {"error": str(e)}

@app.post("/attack")
async def run_attack(
    payload: Dict[str, Any] = Body(...),
    rho_0: float = 1.5,
    d_0_days: float = 30.0,
    phi: float = 1.5,
    use_dates: bool = True,
):
    """Thuc thi tan cong giai an danh dua tren tri thuc bo tro da biet."""
    try:
        aux_info = payload.get("aux_info", [])
        if not aux_info:
            return {"error": "No auxiliary information provided"}

        processed = []
        for item in aux_info:
            rec = {"movie_id": item["movie_id"], "rating": float(item["rating"])}
            if "date" in item and item["date"]:
                rec["timestamp"] = int(pd.to_datetime(item["date"]).timestamp())
            elif "timestamp" in item:
                rec["timestamp"] = int(item["timestamp"])
            else:
                rec["timestamp"] = 883612800 # mac dinh 1998-01-01
            processed.append(rec)

        d_0 = d_0_days * 24 * 3600
        return attack_engine.run_attack(
            processed, rho_0=rho_0, d_0=d_0, phi=phi, use_dates=use_dates
        )
    except Exception as e:
        import traceback
        print(traceback.format_exc())
        return {"error": str(e)}

@app.get("/auto_aux")

```

```

async def auto_aux(
    user_id: int = Query(None, description="ID nạn nhân; bỏ trống = chọn ngẫu nhiên"),
    num_movies: int = 3,
    rating_error: int = 0,
    date_error_days: int = 0,
    prefer_rare: bool = False,
    seed: int = 42,
):
    """
    Sinh "tri thức bẻ gãy" từ một người dùng CŨ THAT, cho phép thêm nhiều đề
    minh họa tính bẻ gãy. Trả về danh sách phim kèm tên đề nạp thẳng vào
    bảng tri thức của kẻ tấn công.
    """
    try:
        engine = get_db_engine()
        if user_id is None:
            import numpy as np
            rng = np.random.RandomState(seed)
            uids = pd.read_sql("SELECT DISTINCT user_id FROM ratings", engine)["user_id"]
            user_id = int(uids.iloc[rng.randint(0, len(uids))])

        aux = attack_engine.sample_aux_from_user(
            user_id, num_movies=num_movies, rating_error=rating_error,
            date_error_days=date_error_days, seed=seed, prefer_rare=prefer_rare,
        )
        for a in aux:
            row = pd.read_sql(
                "SELECT title FROM movies WHERE id = ?", engine, params=(a["movie_id"],)
            )
            a["title"] = row["title"].iloc[0] if not row.empty else f"#{a['movie_id']}"
            a["date"] = pd.to_datetime(a["timestamp"], unit="s").strftime("%Y-%m-%d")
            s, w = attack_engine.get_movie_weight(a["movie_id"])
            a["support"] = s
            a["weight"] = round(w, 4)
        return {"victim_user_id": user_id, "aux_info": aux}
    except Exception as e:
        import traceback
        print(traceback.format_exc())
        return {"error": str(e)}

@app.get("/")
async def root():
    return {
        "message": "De-anonymization Demo API is running",
        "endpoints": ["/stats", "/movies/search", "/attack", "/auto_aux"],
    }

```

Mã nguồn A.3. `experiment.py` - thực nghiệm định lượng tỉ lệ định danh trên 300 nạn nhân

```

"""
Thực nghiệm định lượng trên MovieLens 100k, tại hiện kết quả chính của bài báo
Narayanan & Shmatikov (Fig.4, Fig.8): do tỉ lệ giai an danh thành công theo số

```

luong phim ma ke tan cong biet, trong cac kich ban:

- aux chinh xac, CO ngay (sai so tuy chon)
- aux chinh xac, KHONG ngay
- aux co NHIEU (sai rating +-1, sai ngay +-14) -kiem tra tinh ben vung

Ket qua ghi ra results/deanon_results.csv de ve bieu do trong bao cao.

Mot lan dinh danh tinh la THANH CONG khi eccentricity > phi VA user duoc chon dung la nan nhan that (true positive).

```
"""
import os
import csv
import numpy as np
import pandas as pd
from attack_engine import AttackEngine

RESULTS_DIR = os.path.join(os.path.dirname(__file__), "results")
os.makedirs(RESULTS_DIR, exist_ok=True)

def run_scenario(engine: AttackEngine, user_ids, num_movies, rating_error,
                 date_error_days, use_dates, prefer_rare, phi=1.5, seed0=1000):
    success = 0
    eccs = []
    for k, uid in enumerate(user_ids):
        aux = engine.sample_aux_from_user(
            uid, num_movies=num_movies, rating_error=rating_error,
            date_error_days=date_error_days, seed=seed0 + k, prefer_rare=prefer_rare,
        )
        if len(aux) < num_movies:
            continue
        res = engine.run_attack(aux, phi=phi, use_dates=use_dates)
        if "error" in res:
            continue
        eccs.append(res["eccentricity"])
        idu = res["identified_user"]
        if idu is not None and idu["user_id"] == uid:
            success += 1
    n = len(eccs)
    rate = success / n if n else 0.0
    return rate, (float(np.mean(eccs)) if eccs else 0.0), n

def main(num_users=300, seed=7):
    engine = AttackEngine()
    rng = np.random.RandomState(seed)
    all_users = engine.ratings_df["user_id"].unique()
    # Chi chon nguai da xem du nhieu phim de co the lay 8 phim lam aux
    counts = engine.ratings_df.groupby("user_id").size()
    eligible = counts[counts >= 8].index.values
    sample_users = rng.choice(eligible, size=min(num_users, len(eligible)),
                              replace=False)

    movie_counts = [2, 3, 4, 6, 8]
    scenarios = [
```

```

        ("exact_dates", dict(rating_error=0, date_error_days=0, use_dates=True,
                             prefer_rare=False)),
        ("exact_nodates", dict(rating_error=0, date_error_days=0, use_dates=False,
                                prefer_rare=False)),
        ("noisy_dates", dict(rating_error=1, date_error_days=14, use_dates=True,
                              prefer_rare=False)),
        ("rare_nodates", dict(rating_error=0, date_error_days=0, use_dates=False,
                              prefer_rare=True)),
    ]

    out_path = os.path.join(RESULTS_DIR, "deanon_results.csv")
    rows = []
    print(f"{'scenario':16} {'k':>3} {'rate':>8} {'mean_ecc':>10} {'n':>5}")
    for name, params in scenarios:
        for k in movie_counts:
            rate, mean_ecc, n = run_scenario(
                engine, sample_users, num_movies=k, **params)
            rows.append({"scenario": name, "num_movies": k,
                        "success_rate": round(rate, 4),
                        "mean_eccentricity": round(mean_ecc, 4), "n": n})
            print(f"{name:16} {k:>3} {rate:>8.3f} {mean_ecc:>10.3f} {n:>5}")

    with open(out_path, "w", newline="") as f:
        w = csv.DictWriter(f, fieldnames=["scenario", "num_movies",
                                          "success_rate", "mean_eccentricity", "n"])

        w.writeheader()
        w.writerows(rows)
    print(f"\nĐã ghi {len(rows)} dòng vào {out_path}")

if __name__ == "__main__":
    main()

```

Mã nguồn A.4. demo_attack.py - một phiên tấn công mẫu chạy trên dòng lệnh

```

"""
Demo nhanh tren dong lenh: lay "dau van tay" 4 phim cua mot nguoi dung that
(them nhieu nhe) roi chay thuat toan Scoreboard-RH de giai an danh.
"""

from attack_engine import AttackEngine
import pandas as pd

VICTIM = 200 # mot nguoi dung co that trong MovieLens 100k

eng = AttackEngine()
aux = eng.sample_aux_from_user(VICTIM, num_movies=6, rating_error=0,
                               date_error_days=0, seed=2024, prefer_rare=False)

titles = pd.read_sql("SELECT id, title FROM movies", eng.engine).set_index("id")["title"]
print(f"\n[+] Nan nhan that: user #{VICTIM}")
print("[+] Tri thuc bo tro (aux) ke tan cong nam giu (6 phim, chinh xac):")
for a in aux:
    s, w = eng.get_movie_weight(a["movie_id"])
    print(f"        - {titles.get(a['movie_id'],'?'):42} rating={a['rating']:.0f} "
          f"support={s:<4} wt={w:.3f}")

```

```

res = eng.run_attack(aux, phi=1.5)
idu = res["identified_user"]
print(f"\n[+] Tong so ung vien xet den : {res['total_candidates']}")
print(f"[+] Eccentricity          : {res['eccentricity']:.4f}   (nguong phi=1.5)")
print(f"[+] Top-3 ung vien          :")
for i, c in enumerate(res["top_candidates"][:3]):
    print(f"      #{i+1}   user {c['user_id']:<4}   similarity={c['similarity']:.4f}")

if idu:
    ok = "DUNG" if idu["user_id"] == VICTIM else "SAI"
    print(f"\n[!] DINH DANH THANH CONG -> user #{idu['user_id']}   ({ok} nan nhan)")
    print(f"[!] Da phoi bay {len(res['full_history'])} luot xem phim rieng tu cua nan nhan."
    )
else:
    print(f"\n[-] Khong du eccentricity de dinh danh.")

```